

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20.3

SERVER-SIDE SESSIONS TOKEN MANAGER

 Copy as Markdown

Server-side sessions token manager

The `ServerSideSessionsTokenManager` validates each token against a server-side session, a database record of who is signed in on which device. The database stores only a hash of the session key, never the token itself, which stays in the app. These auth sessions are unrelated to the `Session` object that endpoint methods receive, which is the context of a single request. This token manager provides:

- Stateful authentication (database queries for validation).
- Immediate session revocation.
- Support for session expiration and inactivity timeouts.

WEB APPS

With cookie-based web authentication enabled, browsers receive the session token as an `httpOnly` cookie instead of in the response body.

Server-side configuration

The `ServerSideSessionsTokenManager` is created by passing a `ServerSideSessionsConfig` object in the `tokenManagerBuilders` list of `pod.initializeAuthServices()`:

```
pod.initializeAuthServices(  
  tokenManagerBuilders: [  
    ServerSideSessionsConfig(  
      // The pepper used to hash the session key.  
      sessionKeyHashPepper: pod.getPassword('serverSideSessionKeyHashPepper')!,  
    ),  
  ],  
);
```



Ask AI

TIP

You can use the `ServerSideSessionsConfigFromPasswords` constructor instead of the `ServerSideSessionsConfig` above. It loads the pepper from the `config/passwords.yaml` file or environment variables. It expects the `serverSideSessionKeyHashPepper` key in the file, or the `SERVERPOD_PASSWORD_serverSideSessionKeyHashPepper` environment variable.

Basic configuration options

- `sessionKeyHashPepper`: Required. A secret pepper used for hashing session keys (see [storing secrets](#) for what a pepper is). Must be at least 10 characters long, but [the recommended length is 32 bytes](#).

Extra configuration options

Common configuration options for the `ServerSideSessionsTokenManager`. For more details on all options, check the `ServerSideSessionsConfig` in-code documentation.

```
final serverSideSessionsConfig = ServerSideSessionsConfigFromPasswords(  
  // Optional: Fallback peppers for pepper rotation  
  // This is useful for allowing old sessions to be validated after a rotation.  
  fallbackSessionKeyHashPeppers: [  
    pod.getPassword('oldSessionKeyHashPepper')!,  
  ],  
  // Optional: Set default session lifetime (default is to never expire)  
  defaultSessionLifetime: Duration(days: 30),  
  // Optional: Set inactivity timeout (default is to never timeout)  
  defaultSessionInactivityTimeout: Duration(days: 7),  
  // Optional: Configure session key properties  
  sessionKeySecretLength: 32,  
  sessionKeyHashSaltLength: 16,  
  // Check the [ServerSideSessionsConfig] documentation for more options.  
);
```

Attaching custom metadata to sessions

You can attach custom metadata to each server-side session by providing an `onSessionCreated` callback. This is useful for storing device information, IP address, user agent, or any other data you need to query or display later (for example, in a "sessions" or "devices" list). The callback runs when a session is created, within the same transaction as the session insert.

Define a server-only table that relates to `ServerSideSession` and store your metadata there. Example schema:

```
class: SessionMetadata  
serverOnly: true  
table: session_metadata  
fields:  
  ### The [ServerSideSession] this metadata belongs to
```

```

serverSideSession: module:serverpod_auth_core:ServerSideSession?,
relation(onDelete=Cascade)

### A display name for the session
name: String?

### Device information for the session
deviceName: String?

### IP address from which the session was created
ipAddress: String?

### User agent string
userAgent: String?

indexes:
  server_side_session_id_unique_idx:
    fields: serverSideSessionId
    unique: true

```

Then configure the callback in your server-side sessions config:

```

ServerSideSessionsConfigFromPasswords(
  onSessionCreated:
    (
      final session, {
        required final authUserId,
        required final serverSideSessionId,
        required final transaction,
      }) async {
      await SessionMetadata.db.insertRow(
        session,
        SessionMetadata(
          serverSideSessionId: serverSideSessionId,
          name: 'general-session',
          ipAddress: session.request?.connectionInfo.remote.address.toString(),
          userAgent: session.request?.headers.userAgent,
        ),
        transaction: transaction,
      );
    },
),

```

To revoke sessions based on your custom metadata, query the metadata table for the session IDs you want to revoke and call `revokeToken` for each:

```

final sessionMetadata = await SessionMetadata.db.find(
  session,
  where: (final row) => row.deviceName.equals('Old Device'),
);

for (final row in sessionMetadata) {
  await AuthServices.instance.tokenManager.revokeToken(

```

```

    session,
    tokenId: row.serverSideSessionId.toString(),
  );
}

```

Attaching metadata when creating tokens from an endpoint

The `onSessionCreated` callback is global and runs for every new session, including those created by identity providers. An endpoint that creates a personal access token (PAT) or CLI token often has its own parameters, such as a token name or label, which the callback can't see.

In that case, create the token with `AuthServices.instance.tokenManager.createToken` rather than `issueToken`, which is for [sign-in flows](#). Then use the returned `AuthSuccess.serverSideSessionId` to insert your metadata with the endpoint's parameters:

```

final authSuccess = await AuthServices.instance.tokenManager.createToken(
  session,
  authUserId: userId,
  method: 'pat',
  scopes: {Scope.admin},
);

await SessionMetadata.db.insertRow(
  session,
  SessionMetadata(
    serverSideSessionId: authSuccess.serverSideSessionId,
    name: tokenName, // from your endpoint parameter
    deviceName: deviceName, // from your endpoint parameter
    ipAddress: session.request?.connectionInfo.remote.address.toString(),
    userAgent: session.request?.headers.userAgent,
  ),
);

```

Client-side configuration

The `ServerSideSessionsTokenManager` needs no extra configuration in your app. The client includes the session token in requests automatically. If the session expires or is revoked, the client signs the user out the next time it validates the session. Your app decides what to show next, for example by listening to [authentication state changes](#).

Related

- [Managing tokens](#): issue, validate, revoke, and list tokens.
- [JWT token manager](#): the stateless alternative.
- [Setup](#): where token managers are configured.